

Statistical Learning and Machine Learning

Exercise Session (Week 10)

SOLUTIONS

BOOK EXERCISES

Problem 5.2

Let $\mathbf{x}_n \in \mathbb{R}^D$, $t_n \in \mathbb{R}$, and $\{(\mathbf{x}_n, t_n)\}_{n=1}^N$ denote our observations. The likelihood function of (5.16) is given by:

$$\mathcal{L}(\mathbf{t}|\mathbf{X}, \mathbf{w}, \beta) = \prod_{n=1}^N p(t_n|\mathbf{x}_n, \mathbf{w}) = \prod_{n=1}^N \mathcal{N}(t_n|\mathbf{y}(\mathbf{x}_n, \mathbf{w}), \beta^{-1}\mathbf{I})$$

where $\mathbf{X} = \{\mathbf{x}_1, \dots, \mathbf{x}_N\}$ and $\mathbf{t} = \{t_1, \dots, t_N\}$. The log likelihood function of (5.16) is then given by:

$$\ln(\mathcal{L}(\mathbf{t}|\mathbf{X}, \mathbf{w}, \beta)) = \ln\left(\prod_{n=1}^N \mathcal{N}(t_n|\mathbf{y}(\mathbf{x}_n, \mathbf{w}), \beta^{-1}\mathbf{I})\right) \quad (1)$$

$$= \sum_{n=1}^N \ln(\mathcal{N}(t_n|\mathbf{y}(\mathbf{x}_n, \mathbf{w}), \beta^{-1}\mathbf{I})) \quad (2)$$

$$= \sum_{n=1}^N \ln\left(\frac{1}{(2\pi)^{D/2}} \frac{1}{|\beta^{-1}\mathbf{I}|^{1/2}} \exp\left\{\frac{-1}{2}(t_n - \mathbf{y}(\mathbf{x}_n, \mathbf{w}))^T (\beta^{-1}\mathbf{I})^{-1} (t_n - \mathbf{y}(\mathbf{x}_n, \mathbf{w}))\right\}\right) \quad (3)$$

$$= \frac{-1}{2} \sum_{n=1}^N (t_n - \mathbf{y}(\mathbf{x}_n, \mathbf{w}))^T (\beta\mathbf{I}) (t_n - \mathbf{y}(\mathbf{x}_n, \mathbf{w})) + \underbrace{\sum_{n=1}^N \ln\left(\frac{1}{(2\pi)^{D/2}} \frac{1}{|\beta^{-1}\mathbf{I}|^{1/2}}\right)}_{\text{independent of } \mathbf{w}} \quad (4)$$

$$= \frac{-\beta}{2} \sum_{n=1}^N \|t_n - \mathbf{y}(\mathbf{x}_n, \mathbf{w})\|^2 + \underbrace{\sum_{n=1}^N \ln\left(\frac{1}{(2\pi)^{D/2}} \frac{1}{|\beta^{-1}\mathbf{I}|^{1/2}}\right)}_{\text{independent of } \mathbf{w}} \quad (5)$$

The first term in equation (5) is negative proportional to (5.11). The second term in equation (5) is independent of $\mathbf{w}$, and therefore it does not effect the value of $\mathbf{w}$, when finding the optimal value of $\mathbf{w}$. It only acts as a offset of the value of the log likelihood function. Hence, maximizing Equation (5) w.r.t. $\mathbf{w}$ corresponds to minimizing (5.11) w.r.t. $\mathbf{w}$.

Problem 5.5

Let K be the number of classes of the the multiclass neural network model, N be the number of data points $\mathbf{x} = \{\mathbf{x}_1, \dots, \mathbf{x}_N\}$, and $t_{n,k} \in \{0, 1\}$ be the binary target variables associated with the output. Given the network output interpretation $y_k(\mathbf{x}_n, \mathbf{w}_k) = p(t_{n,k} = 1 | \mathbf{x}_n)$, the conditional distribution of the target vector $\mathbf{t}_n = \{t_{n,1}, \dots, t_{n,K}\}$ of the n th data point $\mathbf{x}_n$ becomes:

$$p(\mathbf{t}_n | \mathbf{w}_1, \dots, \mathbf{w}_K) = \prod_{k=1}^K p(t_{n,k} = 1 | \mathbf{x}_n)^{t_{n,k}} = \prod_{k=1}^K y_k(\mathbf{x}_n, \mathbf{w}_k)^{t_{n,k}}$$

The likelihood function across all datapoints, then becomes:

$$p(\mathbf{T} | \mathbf{w}_1, \dots, \mathbf{w}_K) = \prod_{n=1}^N \prod_{k=1}^K y_k(\mathbf{x}_n, \mathbf{w}_k)^{t_{n,k}}$$

The log likelihood function becomes:

$$\ln(p(\mathbf{T} \mid \mathbf{w}_1, \dots, \mathbf{w}_K)) = \ln \left(\prod_{n=1}^N \prod_{k=1}^K y_k(\mathbf{x}_n, \mathbf{w}_k)^{t_{n,k}} \right) \quad (6)$$

$$= \sum_{n=1}^N \sum_{k=1}^K t_{n,k} \ln(y_k(\mathbf{x}_n, \mathbf{w}_k)) \quad (7)$$

Thus, maximizing (7) is equivalent to minimizing the cross-entropy error function in (5.24).

Problem 5.6

Note: This solution uses (5.23) instead of (5.21), as the former assumes K outputs, whereas the latter only assumes 1 output.

Recap: The logistic sigmoid (4.59) and its derivative (4.88) are given by $\sigma(x) = \frac{1}{1+\exp(-x)}$ and $\frac{d\sigma(x)}{dx} = \sigma(x)(1 - \sigma(x))$.

Let $y_{n,k} = \sigma(a_k)$, the taking the derivative of (5.23) with respect to the k th activation a_k :

$$\frac{dE(\mathbf{w})}{da_k} = \frac{d}{da_k} \left\{ - \sum_{n=1}^N \sum_{c=1}^K t_{n,c} \ln(y_{n,c}) + (1 - t_{n,c}) \ln(1 - y_{n,c}) \right\} \quad (8)$$

$$= - \sum_{n=1}^N \sum_{c=1}^K \frac{d}{da_k} \{ t_{n,c} \ln(y_{n,c}) + (1 - t_{n,c}) \ln(1 - y_{n,c}) \} \quad (9)$$

$$= - \sum_{n=1}^N \frac{d}{da_k} \{ t_{n,k} \ln(y_{n,k}) + (1 - t_{n,k}) \ln(1 - y_{n,k}) \} \quad (10)$$

$$= - \sum_{n=1}^N \left(t_{n,k} \frac{1}{y_{n,k}} \frac{d}{da_k} \{ y_{n,k} \} + (1 - t_{n,k}) \frac{1}{1 - y_{n,k}} \frac{d}{da_k} \{ 1 - y_{n,k} \} \right) \quad (11)$$

$$= - \sum_{n=1}^N \left(t_{n,k} \frac{1}{y_{n,k}} \frac{d}{da_k} \{ y_{n,k} \} + (1 - t_{n,k}) \frac{1}{1 - y_{n,k}} \left(\frac{d}{da_k} \{ 1 \} - \frac{d}{da_k} \{ y_{n,k} \} \right) \right) \quad (12)$$

$$= - \sum_{n=1}^N \left(t_{n,k} \frac{1}{y_{n,k}} y_{n,k} (1 - y_{n,k}) + (1 - t_{n,k}) \frac{1}{1 - y_{n,k}} (0 - y_{n,k} (1 - y_{n,k})) \right) \quad (13)$$

$$= - \sum_{n=1}^N (t_{n,k} (1 - y_{n,k}) - (1 - t_{n,k}) y_{n,k}) \quad (14)$$

$$= - \sum_{n=1}^N (t_{n,k} - t_{n,k} y_{n,k} - y_{n,k} + t_{n,k} y_{n,k}) \quad (15)$$

$$= - \sum_{n=1}^N (t_{n,k} - y_{n,k}) \quad (16)$$

$$= \sum_{n=1}^N (y_{n,k} - t_{n,k}) \quad (17)$$

Hence, the derivative of (5.23) w.r.t. the activation a_k satisfies (5.18) $\frac{dE}{da_k} = y_k - t_k$.

Notes:

- In (10), we used $\frac{d}{da_k} \{ y_c \} = 0$ for $k \neq c$.
- In (11), we used the chain rule to find the derivative of $\ln(y_k(a_k))$ w.r.t. a_k .
- In (13), we used the derivative of the logistic sigmoid (4.88).
- In (14) to (16), we are just simplifying the expression.
- In (17), we move the $-$ from outside the sum to inside the sum.

NEURAL NETWORKS

- 1) Parameters of the neural network are the weights and the biases of its layers $\mathbf{W}_1, \mathbf{W}_2, \mathbf{W}_3, \mathbf{W}_{out}, \mathbf{b}_1, \mathbf{b}_2, \mathbf{b}_3, \mathbf{b}_{out}$. Hyper-parameters are the type of activation functions (and their parameters, if they have) used for each layer, the network architecture.
- 2) See Fig. 1.

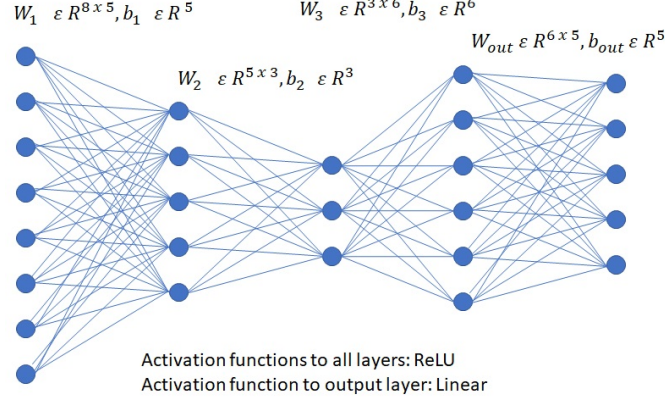


Fig. 1. An MLP mapping 8-d input to 5-d output

- 3) The hyper-parameter values are selected based on the validation error using different neural network realizations (model selection). On each experiment (for a specific network realization) the parameters of the network are estimated by training it using the Backpropagation algorithm to minimize the mean-squared-error between the network's outputs and the target vectors.
- 4) We use the mean-squared-error between the network's outputs and the target vectors. We choose MSE loss because this corresponds to the MLE solution for the network's parameters.
- 5) Because then we have $\mathbf{W} = \mathbf{W}_1 \mathbf{W}_2 \mathbf{W}_3 \mathbf{W}_{out}$, thus we end up to have a linear regression model (after training). Draw two layers (input with 8 neurons and output with 5 neurons) with connections between them (matrix 8×5 plus bias) and linear activation function.
- 6) We would need to (again) use the Backpropagation algorithm to perform iterative optimization. This is because there are many combinations of $\mathbf{W}_k$'s that can lead to the final $\mathbf{W}$ of the two-layer network.
- 7) It would probably be inferior, as optimizing directly for $\mathbf{W}$ using linear regression leads to a global optimum of the MSE criterion. Optimizing with Backpropagation for all $\mathbf{W}_k$'s would lead to one choice for the final $\mathbf{W}$ which may not be the optimal.